

SASD

Lastenheft und Architekturkonzept

Desktop Secret Manager / Tresor-Anwendung
mit mehreren unabhängigen Tresoren, strukturierten Secrets
und zusätzlicher Password-Safe-Interop

Lastenheft und Architekturkonzept

Desktop Secret Manager / Tresor-Anwendung im SASD-Stil

Arbeitsstand fuer die fachliche Freigabe und die anschliessende Ausarbeitung des Pflichtenhefts.

Metadaten	Dokumentstand
Dokumenttyp	Kombiniertes Lastenheft mit vorgelagertem Architekturkonzept
Projektgegenstand	Lokale Windows-Desktop-Anwendung zur Verwaltung verschluesselter Tresore, Passwoerter und strukturierter technischer Secrets
Loesungsrahmen	C# Desktop-Anwendung; Architekturziel: nachvollziehbar, erweiterbar, sicherheitsbewusst und spaeter sauber umsetzbar
Bezugsbasis	Vollstaendiger bisheriger Projektchat einschliesslich Entscheidungen zu Datenmodell, Sicherheit, Dateiverwaltung, Releases und Password-Safe-Interop
Zielorganisation	SASD-Kontext mit Bedarf fuer private, interne, kundenbezogene und providerbezogene Tresore
Status	Fachliche Arbeitsfassung als Grundlage fuer Umsetzungsplanung und anschliessendes Pflichtenheft
Version	1.0
Datum	22.04.2026

Leserhinweis

Dieses Dokument fuehrt zwei Ebenen bewusst zusammen: Zuerst wird das fachliche Zielbild als Lastenheft beschrieben, anschliessend die dazu passende Zielarchitektur. Damit bleibt die fachliche Perspektive vom technischen Entwurf unterscheidbar, zugleich sind die getroffenen Entscheidungen an einer Stelle konsistent dokumentiert.

Management Summary

Gesucht ist keine einfache Passwortliste, sondern eine ruhige, professionell nutzbare Desktop-Anwendung, die mehrere voneinander unabhängige Tresore verwaltet und dabei sowohl klassische Login-Daten als auch strukturierte technische Zugangs- und Kontextinformationen aufnehmen kann. Der Anwendungsfall umfasst insbesondere Hosting-, Datenbank-, FTP-, Mail-, API-, Server- und GitHub-Zugänge sowie weitere Secrets, die im SASD-Umfeld für Betrieb, Entwicklung, Kundenarbeit und interne Organisation relevant sind.

Die Anwendung soll lokal auf Windows laufen, bewusst ohne Cloud-Zwang, ohne Team-Sharing und ohne Browser-Integration in der ersten Ausbaustufe. Sicherheitsfunktionen wie ein Master-Passwort pro Tresor, Auto-Lock, Clipboard-Schutz, atomisches Speichern und verschlüsselte Backups werden als Kern des Produkts verstanden und nicht als spätere Komfortoption.

Für die Interoperabilität mit Password Safe nach Bruce Schneier wird eine additive Strategie festgelegt. Das interne Datenmodell bleibt führend, weil es strukturierte Zusatzfelder, Gruppen, Tags und mehrere technische Eintragstypen sauber abbilden muss. Die Unterstützung von .psafe3 wird daher als Interop-Schicht aufgebaut: in V1 zunächst als kontrollierter Import, in V1.x als Export mit Kompatibilitätsbericht und in V2.0 optional als weiterentwickelter Kompatibilitätsmodus.

Das Dokument zieht die daraus folgenden Konsequenzen für Datenmodell, Sicherheitsgrenzen, Dateiverwaltung, Releaseplanung und Architektur. Es soll verhindern, dass später aus Zeitdruck ein fachlich zu enges, technisch unsauberes oder sicherheitskritisch missverständliches Produkt entsteht.

Kapitelübersicht

- Teil I beschreibt das Lastenheft mit Zielbild, Umfang, Anforderungen, Sicherheitsgrenzen, Releaseplanung und Abnahmekriterien.
- Teil II beschreibt das Architekturkonzept mit Schichtenmodell, Domänenmodell, Sicherheits- und Dateikonzept, Mehrtresor-Strategie und Password-Safe-Interop.
- Beide Teile sind bewusst aufeinander abgestimmt, damit das spätere Pflichtenheft nicht zwischen Fachsicht und Technik zerfällt.

1. Ausgangslage und Problemstellung

Im Projektkontext reicht eine klassische Passwortverwaltung nicht aus. Benötigt wird eine Anwendung, die Zugänge nicht nur als Paar aus Benutzername und Passwort behandelt, sondern als fachlich sinnvolle Einheiten mit technischem und organisatorischem Kontext. Ein Datenbankseintrag benötigt beispielsweise Host, Port, Datenbankname, optional Schema, Hinweise zu SSL oder Zugriffsrolle und gegebenenfalls weitere betriebliche Metadaten. Entsprechendes gilt für Hosting-Zugänge, Mail-Accounts, FTP- oder SFTP-Zugänge, API-Schlüssel, Serverzugriffe oder GitHub-Konten.

Hinzu kommt der Wunsch nach einer belastbaren Ordnungslogik. Zugänge sollen einerseits in festen Gruppen abgelegt werden können, etwa unter SASD GmbH / IONOS / Mail oder SASD GmbH / GitHub, andererseits über Tags flexibel quer auffindbar bleiben, zum Beispiel über IONOS, Produktion, Kunde, GitHub, Datenbank oder Deployment. Ein reines Gruppenmodell wäre zu starr; ein reines Tag-Modell würde zu chaotisch. Das Produkt muss daher beide Ordnungsformen bewusst kombinieren.

Gleichzeitig besteht das fachliche Interesse, bestehende Daten aus Password Safe weiter nutzen zu können. Daraus entsteht die Anforderung, das bekannte .psafe3-Format mindestens zusätzlich zu unterstützen, ohne das eigene Produktmodell auf die Grenzen dieses Formats zu reduzieren. Daraus folgt bereits auf Lastenheft-Ebene eine wesentliche Leitentscheidung: Das Produkt darf sich nicht in einen vermeintlichen 1:1-Klon zwingen lassen, wenn dadurch zentrale Fachanforderungen verlorengehen.

2. Zielbild und Leitidee

Ziel ist eine lokal nutzbare Windows-Desktop-Anwendung, die als persönlicher und unternehmensnaher Tresor für Passwörter, Zugangsdaten, Tokens und strukturierte technische Secrets dient. Die Anwendung soll mehrere voneinander unabhängige Tresore führen können, damit private, SASD-interne, kundenbezogene oder archivierte Informationen sauber getrennt bleiben.

Der fachliche Kern besteht darin, einen einzelnen Zugang vollständig genug zu beschreiben, damit der Eintrag später im Alltag wirklich nützlich ist. Ein Eintrag soll daher nicht nur geheimen Wert und Benutzerkennung enthalten, sondern auch Typ, Gruppe, Tags, beschreibende Notizen und frei definierbare Zusatzfelder. Dadurch wird aus einer Passwortliste ein arbeitsfähiges Secret- und Credential-Repository.

Fest vereinbarte Grundentscheidung

Leitidee: Ein Tresor-Eintrag beschreibt einen primären Zugang oder ein primäres Secret fachlich sauber, nicht nur dessen geheimes Kennwort. Zusammenhänge zwischen mehreren Einträgen entstehen über Gruppen, Tags und gegebenenfalls Referenzen - nicht dadurch, dass unverbundene Dinge in einen einzigen Sammel-Eintrag gepresst werden.

3. Produktgegenstand, Einsatzkontext und Stakeholder

Das Produkt ist als lokale Desktop-Anwendung für Windows vorgesehen. Es soll offline-first arbeiten und ohne zentrale Dienste einsatzfähig sein. Die Hauptnutzung ist Single-User-orientiert. Team-Sharing,

Browser-Autofill, Cloud-Synchronisation oder Multi-Device-Architekturen gehoeren bewusst nicht zum Grundumfang der ersten Ausbaustufen.

Stakeholder sind in erster Linie der kuenftige Hauptnutzer und die SASD-GmbH im fachlichen Sinn, also eine Umgebung, in der sowohl private als auch unternehmensbezogene und projektbezogene Zugaenge sicher, strukturiert und nachvollziehbar verwaltet werden muessen. Daraus folgt ein hohes Gewicht fuer Wartbarkeit, gute Lesbarkeit, nachvollziehbare Bedienung und spaetere Erweiterbarkeit.

Kontext	Einordnung
Nutzungsumgebung	Windows-Desktop, lokaler Einzelplatz, offline-first
Nutzungsarten	Private Tresore, SASD-interne Tresore, providerbezogene Tresore, kundenbezogene Tresore, Archive
Kernobjekte	Tresore, Eintraege, Gruppen, Tags, Zusatzfelder, Interop-Dateien
Hauptnutzen	Sichere Verwaltung, bessere Ordnung, strukturierte Zusatzinformationen, kontrollierte Weiterverwendung bestehender Password-Safe-Daten

4. Fachlicher Leistungsumfang

4.1 Gegenstand der Verwaltung

Die Anwendung soll klassische Logins ebenso verwalten koennen wie Datenbankverbindungen, Server-Zugaenge, Mail-Konten, FTP- und SFTP-Konten, API-Schluesel und weitere Secrets. Darueber hinaus muss sie in der Lage sein, Zusatzinformationen wie Host, Port, Datenbankname, Protokoll, Mailserver, Rollenhinweise, Ablaufdaten, organisatorische Notizen, Verantwortlichkeiten oder projektspezifische Beschreibungen aufzunehmen.

Besonders relevant ist der Wunsch, Zugriffe thematisch und organisatorisch sinnvoll zu strukturieren. Beispiele aus dem Projektkontext sind Eintraege fuer das IONOS-Hosting-Backend, IONOS-Datenbankverbindungen fuer mehrere SASD-Anwendungen, FTP-Zugaenge fuer Webpace-Deployments, GitHub-Organisationen oder einzelne GitHub-Accounts sowie SASD-Mailkonten bei IONOS. Solche Eintraege sollen ueber Gruppen wie SASD GmbH / IONOS / Datenbanken und zusaetzliche Tags wie SASD, IONOS oder Produktion gemeinsam auffindbar sein.

4.2 Bewusste Nicht-Ziele des Grundumfangs

Der Grundumfang schliesst bewusst keine Browser-Integration, keine Cloud-Synchronisation, kein Team-Sharing, keine mobile App, keine browserbasierte Weboberflaeche und keine Plugin-Plattform ein. Ebenfalls nicht Ziel von V1 ist ein vollstaendiger Funktionsgleichstand mit dem gesamten Password-Safe-Oekosystem. Die Anwendung soll kompatibel denken, aber nicht von Anfang an jeden Spezialfall oder jedes historische Detail einer fremden Produktwelt nachbilden muessen.

5. Fachliches Datenmodell

Das Lastenheft legt ein flexibles, aber klar diszipliniertes Datenmodell fest. Zentral ist der Tresor als fachliche Einheit. Ein Tresor enthaelt Gruppen, Eintraege, Tags, fachliche Metadaten und vault-spezifische Einstellungen. Ein Eintrag repraesentiert einen primaeren Zugang oder ein primaeres Secret. Er besitzt mindestens Titel, Typ, Geheimwert beziehungsweise Secret, optional Benutzerkennung, Notizen, genau eine Hauptgruppe, beliebig viele Tags sowie frei definierbare Zusatzfelder.

Zusatzfelder muessen nicht nur Name und Wert kennen, sondern mindestens die fachliche Natur des Felds ausdruecken koennen, etwa Text, Secret, Hostname, Port, URL, E-Mail, Datum, Zahl oder Boolescher Wert. So wird verhindert, dass alle kontextbezogenen Informationen als schwer auswertbarer Notiztext enden.

Fachobjekt	Beschreibung	Pflicht fuer V1
Tresor	Abgeschlossene Verwaltungseinheit mit eigener Datei, eigenem Master-Passwort und eigenen kryptografischen Parametern.	Ja
Eintrag	Primaerer Zugang oder primaeres Secret mit Typ, Titel, geheimem Wert, Gruppe, Tags und Zusatzfeldern.	Ja
Gruppe	Hierarchische Ablage fuer Eintraege; genau eine Hauptgruppe pro Eintrag.	Ja
Tag	Freie querliegende Markierung fuer Suche und Filter.	Ja
Zusatzfeld	Frei definierbares strukturiertes Feld mit Feldart und optional geheimer Behandlung.	Ja
Beziehung	Optionale Referenz zwischen Eintraegen, wenn Zusammenhaenge abgebildet werden sollen.	Spaeter sinnvoll

Wichtig ist die Modellregel, dass ein Eintrag genau einen primaeren fachlichen Gegenstand beschreiben soll. Ein Hosting-Backend, eine FTP-Verbindung und ein Mailkonto koennen organisatorisch zusammengehoeeren, sollen aber als getrennte Eintraege fuehrbar bleiben. Dadurch werden Suche, Maskierung, Export und Wartbarkeit klarer.

6. Funktionale Anforderungen

6.1 Tresorverwaltung

LF-01: Es muessen neue Tresore angelegt, bestehende Tresore geoeffnet, geschlossen und wieder gespeichert werden koennen.

LF-02: Mehrere voneinander unabhaengige Tresor-Dateien muessen moeglich sein, damit private, interne, kundenbezogene oder archivierte Daten getrennt gehalten werden koennen.

LF-03: Jeder Tresor muss mit eigenem Master-Passwort geschuetzt sein und als eigenstaendige fachliche Einheit funktionieren.

LF-04: Die Anwendung soll zuletzt verwendete Tresore lokal wiederfinden koennen, ohne dabei geheime Inhalte oder Master-Passwoerter ausserhalb des Tresors abzulegen.

6.2 Eintragsverwaltung und Zusatzinformationen

LF-05: Eintraege muessen angelegt, bearbeitet, archiviert und geloeschet werden koennen.

LF-06: Eintraege muessen Eintrags Typen wie Login, Database, Mail, Server, FTP/SFTP, API, License, Secure Note und Custom unterstuetzen.

LF-07: Zusatzfelder muessen strukturiert erfasst werden koennen, damit Host, Port, Datenbankname, Mailserver, Protokoll, Rollenhinweis, Ablaufdatum oder aehnliche Informationen nicht nur als Fliesstext hinterlegt werden muessen.

LF-08: Geheime Zusatzfelder muessen als solche markierbar sein und im UI standardmaessig verdeckt behandelt werden.

LF-09: Typische technische Eintragsvorlagen sollen spaeter unterstuetzt werden; in V1 muss das Datenmodell diese Erweiterung bereits sauber aufnehmen koennen.

6.3 Ordnung, Gruppierung, Tags, Suche und Filter

LF-10: Eintraege muessen genau einer Hauptgruppe zugeordnet werden koennen; Gruppen muessen hierarchisch organisierbar sein.

LF-11: Eintraege muessen beliebig viele Tags besitzen koennen, damit providerbezogene, organisatorische oder betriebliche Querverbindungen sichtbar bleiben.

LF-12: Suche und Filter muessen Titel, Gruppen, Tags, Notizen, Typen und Zusatzfelder durchsuchen koennen.

LF-13: Die Bedienung muss sowohl eine Navigation ueber den Gruppenbaum als auch schnelles Wiederfinden ueber Suche und Tags unterstuetzen.

6.4 Sicherheitsbezogene Bedienfunktionen

LF-14: Geheime Werte muessen standardmaessig verborgen sein und nur kontrolliert angezeigt werden koennen.

LF-15: Ein Kopieren geheimer Werte in die Zwischenablage muss unterstuetzt werden; die Zwischenablage ist anschliessend automatisiert zu leeren.

LF-16: Die Anwendung muss sich nach Inaktivitaet automatisch sperren koennen und danach erneut das Master-Passwort des aktiven Tresors verlangen.

LF-17: Ein Passwortgenerator muss bereitgestellt werden, um neue starke Kennwoerter in der Anwendung erzeugen zu koennen.

6.5 Dateiverwaltung, Backups und Wiederherstellung

LF-18: Speichervorgaenge duerfen bestehende Tresor-Dateien nicht durch halbfertige oder unterbrochene Schreibvorgaenge unbrauchbar machen.

LF-19: Verschlüsselte Backups der Tresore sollen unterstuetzt werden, damit Wiederherstellung nach Bedienfehlern oder Dateischaeden moeglich bleibt.

LF-20: Parallele Oeffnungen desselben Tresors muessen kontrolliert und fuer V1 konservativ behandelt werden.

6.6 Interoperabilitaet mit Password Safe

LF-21: Das Produkt soll das dokumentierte Password-Safe-Format .psafe3 mindestens zusaetzlich unterstuetzen, ohne das interne Produktmodell darauf zu reduzieren.

LF-22: In V1 soll ein kontrollierter Import aus .psafe3 moeglich sein, bei dem nicht direkt abbildbare Inhalte sichtbar markiert oder in nachvollziehbarer Form uebernommen werden.

LF-23: Ein Export in Richtung .psafe3 soll erst folgen, wenn Mapping-Regeln, Verlusthinweise und Kompatibilitaetsberichte sauber definiert sind.

7. Nichtfunktionale Anforderungen und Qualitätsziele

Neben den funktionalen Anforderungen ist vor allem die Qualität des Produkts entscheidend. Die Anwendung muss ruhig, nachvollziehbar und wartbar sein. Bedienflächen sollen technisch-professionell wirken, aber nicht überladen. Die Daten müssen langfristig migrierbar und die Architektur erweiterbar bleiben.

Qualitätsziel	Bedeutung	Priorität
Sicherheit	Schutz der Tresor-Dateien und sichere Standardbedienung haben Vorrang vor Komfortabkürzungen.	Sehr hoch
Nachvollziehbarkeit	Datenmodell, Bedienung und spätere Implementierung sollen klar und dokumentierbar bleiben.	Sehr hoch
Wartbarkeit	Code und Architektur sollen bewusst sauber aufgebaut und später gut kommentierbar bleiben.	Sehr hoch
Erweiterbarkeit	TOTP, Anhaenge, weitere Interop-Funktionen oder Templates sollen später ohne Bruch aufsetzbar sein.	Hoch
Performance	Lokale Alltagsnutzung soll flüssig bleiben; extreme Optimierung steht jedoch hinter Sicherheit und Klarheit zurück.	Mittel

8. Sicherheitsgrenzen und Schutzmodell

Das Produkt ist als sicherheitskritische Einzelplatz-Anwendung zu betrachten. Es soll wirksam gegen Verlust oder Diebstahl der Tresor-Datei, gegen neugierige Mitbenutzer am Gerät und gegen unbeabsichtigte Offenlegung durch schlechte Bedienprozesse schützen. Dazu gehören Master-Passwort, verschlüsselte Tresor-Dateien, verdeckte Anzeige geheimer Werte, Auto-Lock, Clipboard-Schutz und eine strikte Trennung zwischen Tresor-Daten und lokalen App-Einstellungen.

Nicht versprochen wird ein vollständiger Schutz gegen bereits kompromittierte Systeme. Malware, Keylogger, kompromittierte Benutzerkonten, RAM-Auslese oder andere Formen eines bereits übernommenen Windows-Systems liegen ausserhalb des wirksam beherrschbaren Schutzmodells. Das Lastenheft verlangt daher ehrliche Dokumentation dieser Grenze und verbietet missverständliche Sicherheitsversprechen.

Verbindliche Sicherheitslinie

Sicherheitsprinzipien fuer das Produkt: kein Klartext-Export ohne deutliche Warnung, keine Geheimnisse in Logdateien, keine Master-Passwoerter ausserhalb der Tresoroeffnung, keine versteckte Recovery-Hintertuer und keine stillschweigende Preisgabe nicht kompatibler Daten bei Import oder Export.

9. Dateiverwaltung fuer mehrere unabhaengige Tresore

Mehrere Tresore sind ausdruecklicher Bestandteil des Zielbilds. Fachlich bedeutet dies, dass private, SASD-interne, providerbezogene, kundenbezogene und archivierte Informationen getrennt gespeichert, getrennt

entsperrt und getrennt gesichert werden koennen. Ein Tresor ist daher nicht nur eine Ansicht im Programm, sondern eine eigenstaendige Datei mit eigener Schutzdomaene.

Fuer V1 wird festgelegt, dass viele Tresor-Dateien parallel im Produktkontext existieren duerfen, aber pro Programmfenster genau ein aktiver schreibbarer Tresor vorgesehen ist. Dadurch bleibt die Handhabung robust und Konfliktfaelle werden reduziert. Spaetere Releases koennen read-only-Zweitnutzung, globalere Uebersichten oder Vergleichsfunktionen nachziehen, ohne das Grundmodell zu brechen.

Tresor- und Dateiverwaltung

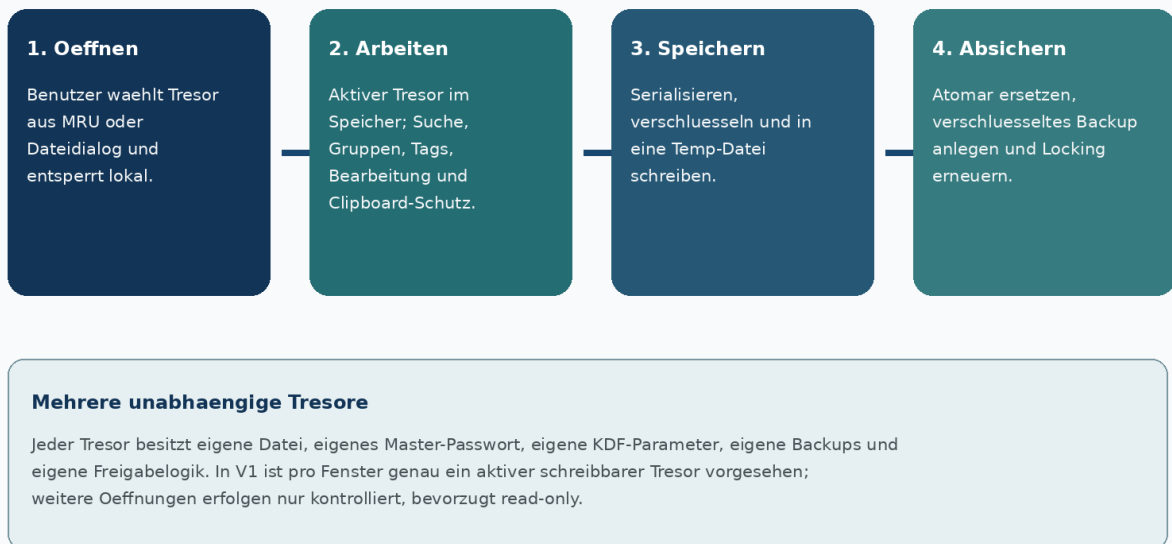


Abbildung 1 - Fachliche Sicht auf Oeffnen, Arbeiten, Speichern und Absichern eines Tresors.

10. Releaseplanung

Die Releaseplanung staffelt bewusst zwischen tragfaehigem Fundament, kontrollierter Haertung und spaeteren Komfort- und Kompatibilitaetsfunktionen.

Release	Inhalt	Bewusste Abgrenzung
V1	Eigenes .svault-Format, mehrere Tresore, Gruppen, Tags, Typen, Zusatzfelder, Suche, Generator, Auto-Lock, Clipboard-Clear, Backups und .psafe3-Import.	Kein .psafe3-Export, keine Anhaenge, kein TOTP, kein Browser-Autofill und kein Team-Sharing.
V1.x	Haertung, Konfliktbehandlung, Templates, erweiterte Migrationen, read-only-Strategien und .psafe3-Export mit Bericht.	Noch kein voller Kompatibilitaetsmodus und keine cloubasierte Architektur.
V2.0	Weiterentwickelte Interop, Kompatibilitaetsmodus, Multi-Vault-Suche, Cross-Vault-Referenzen und TOTP.	Die Grundlinie bleibt lokaler Tresor ohne Cloud- oder Team-Zwang.

11. Abnahmekriterien fuer V1

Die Abnahme von V1 erfolgt ueber pruefbare Nutzungsergebnisse und nicht nur ueber Menuepunkte. Erfolgreich ist V1 dann, wenn die Anwendung den definierten Kernnutzen im Alltag bereits sicher und nachvollziehbar erfuehlt.

- Es laesst sich mindestens ein neuer Tresor anlegen, speichern, schliessen und erneut oeffnen, ohne dass Daten verloren gehen.
- Es lassen sich mehrere voneinander getrennte Tresore fuehren, ohne dass ihre Inhalte vermischt oder gemeinsam entsperrt werden.
- Eintraege koennen mit Typ, Gruppe, Tags und strukturierten Zusatzfeldern fuer typische technische Anwendungsfaelle sinnvoll erfasst werden.
- Suche und Filter finden Eintraege anhand von Titeln, Gruppen, Tags, Notizen und Zusatzfeldern innerhalb des aktiven Tresors wieder.
- Geheime Werte werden standardmaessig verborgen, lassen sich kontrolliert kopieren und werden anschliessend wieder aus der Zwischenablage entfernt.
- Nach Inaktivitaet sperrt sich die Anwendung und verlangt erneut das Master-Passwort des aktiven Tresors.
- Ein Import aus .psafe3 ist moeglich und macht sichtbar, wenn Informationen nicht verlustfrei uebernommen werden konnten.
- Lokale Einstellungen enthalten keine geheimen Nutzdaten und keine Master-Passwoerter.

12. Risiken, Annahmen und bewusste Abgrenzungen

Das Lastenheft geht davon aus, dass der Einsatz auf einem grundsaeztlich vertrauenswuerdigen Windows-System erfolgt. Wird diese Annahme verletzt, sinkt die praktische Schutzwirkung des Produkts erheblich. Als weiteres Risiko gilt eine zu starre Ausrichtung am Fremdformat .psafe3; sie wuerde die benoetigten Zusatzinformationen und Organisationsfunktionen unnoetig beschneiden.

Bewusst abgegrenzt werden fuer die Grundausbauetufen Browserintegration, Cloud-Synchronisation, Multi-User-Berechtigungen, Plugin-Mechanismen und mobile Clients. Diese Themen koennen spaeter erneut bewertet werden, gehoeren aber nicht in das fachliche Fundament, auf dem das Pflichtenheft zuerst aufbauen soll.

Ebenso wird bewusst auf eine unredliche Gleichsetzung mit einem vollumfaenglichen 1:1-Nachbau von Password Safe verzichtet. Das Ziel ist ein eigenstaendiges, gut dokumentiertes Produkt mit zusaetzlicher Interop, nicht eine unsauber erkaufte Scheingleichheit.

Teil II - Architekturkonzept

Zielarchitektur, Modulstruktur, Sicherheitskonzept, Dateiformat und Interop-Strategie

13. Architekturziele und Leitlinien

Die Architektur soll das fachliche Zielbild tragen, ohne es unnötig zu verkomplizieren. Sie muss deshalb lokal, robust und schrittweise ausbaubar sein. Sicherheit, Nachvollziehbarkeit und spätere Erweiterbarkeit haben Vorrang vor kurzfristiger Bequemlichkeit.

Im SASD-Stil bedeutet dies konkret: ruhige, klar gegliederte Strukturen; bewusst getrennte Schichten; ein kleines, starkes Kernmodell; technische Ehrlichkeit bei Sicherheit und Interop; sowie Dokumentation, die spätere Pflichtenhefte, Implementierung und Wartung unterstützt.

Leitentscheidung

Architekturleitlinie: Das interne Produktmodell ist führend. Kryptografie, Dateiformat und Password-Safe-Interop werden darum herum organisiert - nicht umgekehrt.

14. Zielarchitektur im Ueberblick

Die Zielarchitektur trennt Oberfläche, Anwendungslogik, Domänenmodell sowie Sicherheits- und Speicherverantwortung. Für die erste Umsetzungsphase wird eine .NET-8-Desktop-Anwendung mit WinForms als pragmatische und gut nachvollziehbare UI-Basis empfohlen. Die Architektur muss jedoch so strukturiert sein, dass Sicherheit, Dateiverarbeitung und Interop nicht in UI-Formulare auslaufen.

Zielarchitektur

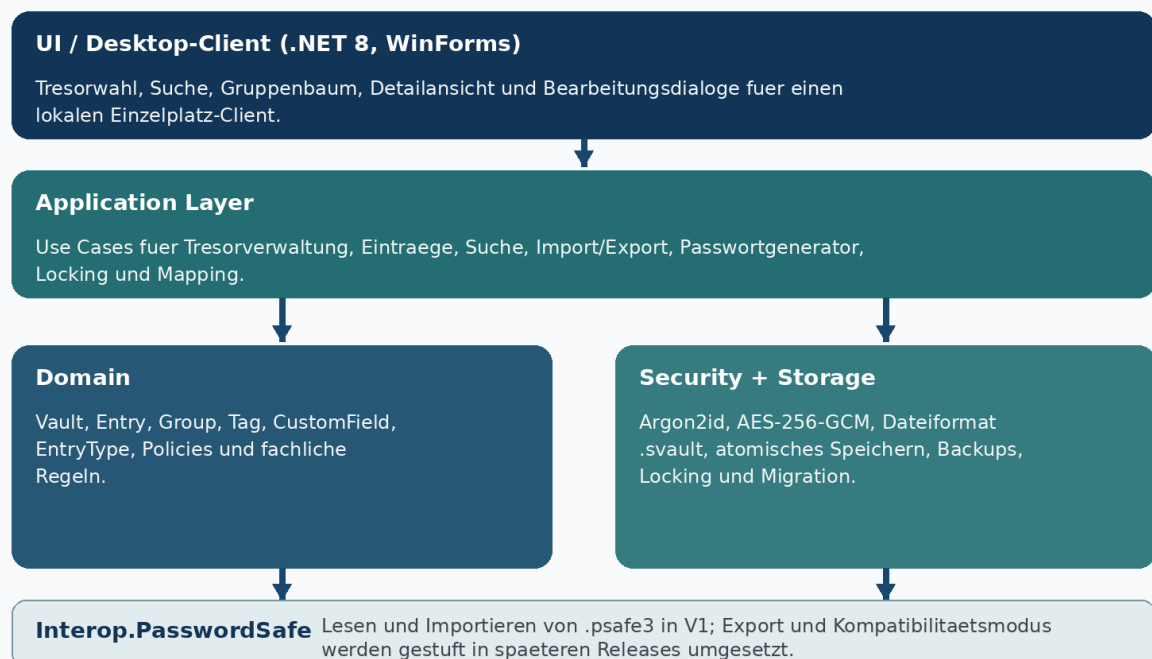


Abbildung 2 - Zielarchitektur mit klarer Trennung von UI, Application Layer, Domain sowie Security, Storage und Password-Safe-Interop.

15. Modul- und Schichtenmodell

Die Anwendung wird in fachlich saubere und klar benannte Module aufgeteilt. Die UI kennt keine kryptografischen Details. Die Storage-Schicht kennt keine konkreten Formulare. Die Interop-Schicht fuer Password Safe darf nicht die Gesamtarchitektur dominieren, sondern arbeitet als Adapter zwischen Fremdformat und internem Modell.

Modul	Verantwortung	Bemerkung
UI	WinForms-Oberflaeche mit Tresorwahl, Gruppenbaum, Suchflaeche, Ergebnisliste, Detailansicht und Dialogen.	Nur Bedienlogik, keine Kryptografie.
Application	Use Cases wie Tresor oeffnen, Eintrag speichern, suchen, importieren, exportieren, sperren, Generator aufrufen.	Orchestriert zwischen UI und Kern.
Domain	Kernobjekte und fachliche Regeln fuer Tresore, Eintraege, Gruppen, Tags und Zusatzfelder.	Fuehrendes Modell.
Security	KDF, Verschluesselung, Clipboard-Schutz, Locking, Redaction-Regeln fuer Logs.	Sicherheitskritischer Kern.
Storage	Lesen, Schreiben, Backups, atomisches Speichern, Lock-Dateien, Migrationen.	Besitzt das Primärformat .svault.
Interop.PasswordSafe	Reader, spaeter Writer, Mapper, Import-Assistent und Kompatibilitaetsberichte.	Additive Schicht.
Tests	Unit-, Integrations-, Interop- und Negativtests.	Pflichtbestandteil.

16. Domänenmodell und fachliche Kernobjekte

Das Domänenmodell muss die im Lastenheft definierten Fachobjekte direkt widerspiegeln. Es soll klein genug bleiben, um verstaendlich zu sein, aber flexibel genug, um technische Anwendungsfaelle abzubilden. Der Tresor ist die fachliche Huelle, der Eintrag das zentrale Arbeitsobjekt.

Kernobjekt	Architektonische Bedeutung
Vault	Wurzelobjekt mit Metadaten, Gruppen, Eintraegen, Tags, lokalen Policies und Referenzen auf KDF-/Formatparameter.
Entry	Zentraler Nutzdatensatz mit ID, Typ, Titel, Secret, optionalem Principal, Gruppe, Tags, Notizen und Zusatzfeldern.
EntryType	Fachlicher Typisierer fuer Login, Database, Mail, Server, FTP/SFTP, API, License, Secure Note oder Custom.
Group	Hierarchische Struktur fuer Ablage und Navigation.
Tag	Frei vergebbare Querverbindung fuer Suche, Filter und semantische Mehrfachzuordnung.
CustomField	Strukturiertes Zusatzfeld mit Name, Wert, Feldart, Geheimkennzeichnung und Sortierreihenfolge.

Der Eintrag wird bewusst als primaere fachliche Einheit behandelt. Verwandte Zugangsdaten bleiben getrennte Eintraege und werden ueber Gruppen, Tags oder spaetere Referenzen zusammengefuehrt. Dadurch werden Interop, Suche, UI und Export ehrlicher und stabiler.

17. Sicherheitskonzept

Die Architektur setzt fuer V1 auf einen starken, aber realistisch dokumentierten lokalen Sicherheitsansatz. Vorgesehen sind AES-256-GCM fuer die Datenverschluesselung und Argon2id fuer die Ableitung des Schluessels aus dem Master-Passwort. Pro Tresor existieren eigene KDF-Parameter und eigene kryptografische Metadaten. Pro Speichervorgang wird eine neue Nonce verwendet.

Die Anwendung trennt strikt zwischen geheimen Tresor-Inhalten und nicht geheimen lokalen App-Einstellungen. Ausserhalb des Tresors duerfen etwa zuletzt verwendete Dateipfade, Fenstergeometrien oder UI-Vorlieben liegen, nicht aber Secrets, Tokens oder Master-Passwoerter. Fehler- und Diagnoselogik muessen Redaction-Regeln befolgen, damit keine geheimen Inhalte in Logdateien oder Ausnahmebehandlungen auftauchen.

Im laufenden Betrieb muessen sensible Werte standardmaessig verborgen angezeigt werden. Der Arbeitsspeicher kann in einer verwalteten .NET-Umgebung nicht absolut gegen alle Szenarien gehaertet werden; daher folgt die Architektur einem Best-Effort-Ansatz: Klartext nur so kurz wie noetig, UI-Felder beim Sperren leeren, keine unnoetigen Duplikate und keine debug-orientierte Preisgabe geheimer Inhalte.

Konsequenz aus dem Schutzmodell

Die Architektur sieht keine Recovery-Hintertuer vor. Wer ein Tresor-Master-Passwort verliert, darf nicht ueber einen versteckten Hersteller- oder Entwicklerweg wieder an die Daten gelangen. Sicherheit geht hier vor Komfort.

18. Vault-Dateikonzept und Persistenz

Das interne Dateiformat wird als .svault gefuehrt. Der Name ist dabei weniger wichtig als die klare Rollenverteilung: Das Format ist das Primärformat des Produkts, nicht bloss ein technisches Detail. Es muss Header-Informationen, Formatversion, KDF-Parameter, kryptografische Metadaten und den verschluesselten Nutzdatenbereich sauber voneinander trennen.

Speichern erfolgt stets kontrolliert: Zunaechst wird der Tresor serialisiert, dann verschluesselt, anschliessend in eine temporaere Datei geschrieben, vollstaendig geflusht und erst danach atomar an Stelle der alten Datei gesetzt. Optional oder standardmaessig konfigurierbare verschluesselte Backup-Dateien ergaenzen diesen Prozess.

Migrationen muessen von Anfang an eingeplant werden. Das Dateiformat benoetigt daher eine Versionsstrategie, damit spaetere Erweiterungen wie TOTP, Anhaenge, neue Feldarten oder veraenderte Policies moeglich bleiben, ohne vorhandene Dateien unkontrolliert zu brechen.

19. Mehrtresor-Konzept

Mehrere unabhaengige Tresore sind kein Nebenschauplatz, sondern ein zentraler Architekturfall. Jeder Tresor besitzt eigene Datei, eigene KDF-Parameter, eigenes Master-Passwort und eigene Backup-Kette. Die Anwendung fuehrt lokal lediglich einen nicht geheimen Katalog zuletzt genutzter Tresore.

Fuer V1 wird festgelegt, dass pro Programmfenster genau ein aktiver schreibbarer Tresor bearbeitet wird. Wird dieselbe Datei parallel erneut geoeffnet, soll die Architektur konservativ reagieren, etwa durch Lock-Datei, Betriebssystem-Sperre oder read-only-Fallback. Erst in spaeteren Releases werden komfortablere Vergleichs-, Kopier- oder Suchszenarien ueber mehrere geoeffnete Tresore hinweg angestrebt.

Diese Entscheidung reduziert Konfliktrisiken und macht Speichervorgaenge, Sperrlogik und Fehlerfaelle deutlich beherrschbarer. Gleichzeitig bleibt die Richtung offen fuer spaetere Multi-Vault-Uebersichten.

20. UI-Konzept

Die Oberflaeche soll ruhig, technisch-professionell und alltagsgeeignet sein. Empfohlen wird ein Hauptfenster mit klaren Zonen: links der Gruppenbaum, oben Suchfeld und Filter, mittig die Liste der Eintraege des aktiven Kontextes und rechts oder unten die Detailansicht. Bearbeitung erfolgt ueber einen dedizierten Dialog oder ein klar abgegrenztes Bearbeitungs-Panel.

Geheime Werte werden verdeckt angezeigt und ueber explizite Aktionen sichtbar oder kopierbar gemacht. Zusatzfelder muessen frei, aber kontrolliert bearbeitbar sein. Der Eintragstyp steuert, welche Feldvorschlaege oder Templates angeboten werden. Dadurch bleibt die Bedienung flexibel, ohne in Beliebigkeit zu kippen.

Die UI soll Tastaturbedienung, Kontextmenues und schnelles Kopieren typischer Werte wie Benutzername, Passwort, Host oder API-Token unterstuetzen. Sicherheit und Bedienbarkeit muessen zusammen gedacht werden, nicht gegeneinander.

21. Suche, Filter und Eintragsvorlagen

Weil die Anwendung mehr als nur Web-Logins aufnehmen soll, ist die Such- und Filterlogik ein Kernbestandteil der Architektur. Die Suchindizierung beziehungsweise Suchauswertung muss Titel, Typ, Gruppe, Tags, Notizen und Zusatzfelder einbeziehen. Nur so wird die Ablage im Alltag tatsaechlich wiederfindbar.

Fuer spaetere Erweiterungen sind fachliche Eintragsvorlagen sinnvoll, etwa fuer Database, Mail, FTP/SFTP oder API. Die Architektur muss diese Templates bereits in V1 mitdenken, auch wenn die erste Version nur das flexible Datenmodell und grundlegende Editorlogik liefert. Templates duerfen das freie Modell nicht ersetzen, sondern sollen es ergonomisch zugreifbar machen.

22. Password-Safe-Interop-Konzept

Die Interop-Schicht zu Password Safe wird stufenweise aufgebaut. In V1 steht ein Reader samt Import-Assistent im Vordergrund. Dieser liest .psafe3-Dateien ein, validiert die Datei, mappt bekannte Standardfelder in das interne Modell und dokumentiert, welche Inhalte nicht direkt abbildbar waren.

Mapping-Regeln werden in drei Klassen gedacht. Erstens direkt abbildbare Informationen wie Titel, Passwort, Benutzername, Gruppe, URL oder Notizen. Zweitens strukturiert abbildbare Informationen, die in Zusatzfelder ueberfuehrt werden koennen. Drittens nur verlustbehaftet oder als importierte Metadaten uebernehmbare Informationen. Wichtig ist, dass solche Faelle nie stillschweigend verschwinden.

In V1.x folgt ein Writer fuer den klar kompatiblen Teil, gekoppelt an einen Kompatibilitaetsbericht. In V2.0 kann darauf ein expliziter Kompatibilitaetsmodus aufsetzen, der bewusst zwischen internem Primärformat und fremdformatnaher Bearbeitung unterscheidet.

Architectural Decision

Interop-Strategie: erst lesen und ehrlich importieren, dann kontrolliert exportieren, erst spaeter Roundtrip-Komfort. So bleibt das eigene Produktmodell stark und die Kompatibilitaet nachvollziehbar.

23. Test- und Qualitaetssicherungskonzept

Die Qualitaetssicherung ist integraler Teil der Architektur. Kryptografie, Dateischreiben, Migrationen, Mapping-Regeln und Suchlogik duerfen nicht ausschliesslich ueber manuelle UI-Tests abgesichert werden. Die Architektur verlangt deshalb automatische Tests auf mehreren Ebenen.

- Unit-Tests fuer KDF, Verschluesselung, Entschluesselung, Serialisierung, Feldtypen, Suchlogik und Mapper.
- Integrationstests fuer Anlegen, Speichern, Schliessen, Wieder-Oeffnen, Locking und Backups.
- Interop-Tests fuer .psafe3-Import, spaeter Export und Kompatibilitaetsberichte.
- Negativtests fuer falsches Master-Passwort, beschaedigte Dateien, abgeschnittene Daten, unbekannte Formatversionen und Konfliktfaelle.

Zusaetzlich ist eine saubere Entwicklerdokumentation wichtig. Da das spaetere C#-Projekt bewusst stark kommentiert werden soll, muss die Architektur Codeorganisation, XML-Kommentare und klare Modulgrenzen foerdern, statt undurchsichtige technische Abkuerzungen zu beguenstigen.

24. Technologie- und Bereitstellungskonzept

Fuer die erste Umsetzungsphase wird eine .NET-8-Anwendung mit WinForms empfohlen. Dies ist fuer einen lokalen Desktop-Secret-Manager ein pragmatischer, nachvollziehbarer und gut wartbarer Ansatz. Die Wahl dient nicht dem optischen Selbstzweck, sondern der schnellen, robusten Bereitstellung einer klaren Windows-Oberflaeche.

Die Bereitstellung kann zunaechst klassisch als lokale Desktop-Installation oder portable Paketierung gedacht werden. Wichtig ist, dass keine geheimen Inhalte in Konfigurationsdateien, Telemetrie oder Update-Mechanismen abfliessen. Zukunftsthemen wie Cross-Platform-Varianten, WPF, MAUI oder Weboberflaechen sind bewusst nicht Teil dieses Konzepts.

25. Architekturentscheidungen nach Release-Stufen

Stufe	Architekturentscheidungen	Wirkung
V1	Primärformat .svault, ein schreibbarer Tresor pro Fenster, Import aus .psafe3, starke Trennung der Schichten.	Sicheres Fundament und gute Nachvollziehbarkeit.
V1.x	Writer und Kompatibilitaetsberichte, bessere Konfliktbehandlung, Migrationen, Vorlagen und read-only-Faelle.	Härtung und kontrollierte Interop-Erweiterung.
V2.0	Kompatibilitaetsmodus, Multi-Vault-Suche, Cross-Vault-Referenzen und weitere Sicherheits-/Komfortfunktionen.	Reiferes Produkt ohne Bruch der Grundarchitektur.

26. Offene, aber bewusst vertagte Themen

Einige Themen sind fachlich interessant, werden jedoch absichtlich nicht in das Grundkonzept hineingezogen. Dazu gehoeren TOTP, echte Binäaraanhaenge, Browser-Autofill, Cloud-Sync, Team-Sharing, mobile Clients und Plugin-Modelle. Diese Faelle koennen spaeter nur dann sinnvoll ergaenzt werden, wenn das Kernmodell, die Sicherheitslinie und die Dateiverwaltung bereits stabil sind.

Die Architektur dokumentiert diese Themen daher nicht als Luecke, sondern als bewusst vertagte Erweiterungsfelder. Das schafft Planungssicherheit fuer das nachfolgende Pflichtenheft.

27. Schlussbemerkung

Das vorliegende Dokument fasst den fachlichen und architektonischen Entscheidungsstand fuer einen Desktop Secret Manager im SASD-Stil zusammen. Es schafft einen belastbaren Rahmen fuer die anschliessende Erstellung des Pflichtenhefts und fuer eine spaetere Umsetzung in C# als saubere, gut kommentierte Windows-Desktop-Anwendung.

Besonders wichtig ist, dass die vier kritischen Grundfragen jetzt konsistent geklaert sind: das saubere interne Datenmodell, ehrliche Sicherheitsgrenzen, eine robuste Dateiverwaltung fuer mehrere unabhaengige Tresore und eine realistische Interop-Strategie mit Password Safe. Genau diese Entscheidungen machen aus einem bloessen Clone-Vorhaben ein tragfaehiges Produktkonzept.

Anhang A - Zusammenfassung der verbindlichen Grundentscheidungen

- Das interne Datenmodell bleibt fuehrend.
- Das Produkt besitzt ein eigenes Primärformat .svault.
- .psafe3 wird additiv ueber eine Interop-Schicht unterstuetzt.
- Ein Eintrag beschreibt einen primaeren Zugang oder ein primaeres Secret.
- Gruppen und Tags sind beide Pflichtbestandteile der Ordnungslogik.
- Mehrere unabhaengige Tresore gehoeren ab V1 zum Produktkern.
- Pro Fenster ist in V1 genau ein aktiver schreibbarer Tresor vorgesehen.
- Sicherheit vor Komfort, aber ohne unrealistische Heilsversprechen.
- Pflichtenheft und Implementierung bauen auf diesem Rahmen auf.